

Experimental Documentation

Analysis of Network Latency Impact on MAPPO in the RWARE Environment

1. Introduction and Research Objective

In modern industrial scenarios — such as automated logistics and drone fleet management — operational efficiency relies on the real-time cooperation of dozens of robots. However, the transition from idealized simulations to real-world deployments inevitably encounters a fundamental obstacle: hardware limitations and Wi-Fi congestion introduce temporal latencies that distort the perception of reality for autonomous systems.

This document constitutes the technical and scientific documentation of the experiment conducted as part of the Network Intelligence course project (Communications Engineering). The central objective of this study is to scientifically demonstrate how standard Multi-Agent Reinforcement Learning (MARL) algorithms, despite achieving optimal performance under ideal simulation conditions, undergo critical and structural degradation in the presence of communication delays — a condition inherent to any real distributed control system, particularly in mobile networks and cyber-physical systems.

The choice of the RWARE (Robot Warehouse) environment as the experimental testbed is deliberate: it is a cooperative multi-agent environment in which two robots must coordinate their actions to deliver packages to the correct locations. The environment demands tight cooperation and precise temporal planning, making it particularly sensitive to any perturbation in the information flow between agents and environment. The algorithm selected for this study is MAPPO (Multi-Agent Proximal Policy Optimization), one of the most advanced centralized Actor-Critic architectures in the MARL landscape, leveraging a Centralized Critic to stabilize the learning of decentralized actors.

The research hypothesis is as follows: the injection of an artificial observation delay (lag) of 2 steps — simulating the network latency typical of a distributed control system — measurably and documentably impairs both the convergence speed of the algorithm and the long-term stability of the learned policies. The experimental results, as detailed in the subsequent sections, confirm this hypothesis unambiguously through three distinct and interrelated effects: intrinsic logistical inefficiency, informational blindness of the Critic, and structural fragility of the optimized policies.

As future developments, the project also explored two potential mitigation strategies: a Lag-Aware Critic (an architectural modification of `centralV.py` to make the Critic aware of the delay) and a Multi-Head Attention module integrated into the recurrent agent (`rnn_agent.py`), aimed at equipping the system with selective memory over historical observations. The preliminary results of these interventions, while promising, fall outside the scope of the main analysis presented here and are proposed as directions for future research.

2. Background and Reference System

To understand the impact of network latency, it is necessary to outline the two pillars underpinning the reference system under ideal conditions: the logistics simulation environment and the multi-agent learning algorithm.

2.1 The Simulation Environment: Robot Warehouse (RWARE)

The experimental testbed is RWARE, a discretized graphical environment that simulates the operational dynamics of an automated logistics warehouse.

- **Game Dynamics:** The system features a fleet of cooperative agents (robots) whose objective is to locate specific shelves, attach to them, transport them to a designated delivery line, and subsequently reposition the empty shelf.
- **Reward Mechanism:** The system adopts an extremely sparse reward structure. Agents receive no positive feedback for intermediate movements, but obtain a collective reward (equal to +1) exclusively at the precise moment a delivery is successfully completed.
- **Physical Constraints:** Episodes have a fixed and rigid duration of 500 time steps. Agents must navigate the space while avoiding collisions or deadlock situations, which would undermine their temporal efficiency.

2.2 The Control Algorithm: MAPPO

The management and training of the robots relies on MAPPO (Multi-Agent Proximal Policy Optimization), a state-of-the-art algorithm that extends PPO to multi-agent scenarios following the CTDE paradigm (Centralized Training with Decentralized Execution).

- **The Actor (Decentralized Execution):** During execution, each individual agent is guided by a local Actor network. The Actor makes decisions based exclusively on its own partial observation of the environment (the surrounding field of view of the robot), ensuring system scalability.
- **The Critic (Centralized Training):** During training, a second global neural network — the Critic — intervenes. The Critic is an omniscient entity with access to the global state of the entire warehouse (the positions of all robots and all shelves). Its task is to evaluate the quality of the actions taken by the actors and to guide the update of their synaptic weights through value function estimation.

3. Experimental Setup

3.1 Environment and Algorithm

All experiments were conducted using the EPyMARL framework, an open-source Python library designed for MARL research, on the rware-tiny-2ag-v2 variant of the RWARE environment — the minimal 2-agent configuration, chosen to precisely isolate system behaviour by minimising confounding variables. The training horizon was set to 40 million steps to ensure complete observation of long-term convergence and

potential late-stage instability phenomena. All metrics were tracked in real time via Weights & Biases (W&B) on the Progetto-MAPPO-RWARE project.

3.2 Experimental Configurations

To ensure the scientific validity of the comparison, the two configurations differ exclusively in a single parameter — the presence or absence of the lag — while keeping all other hyperparameters identical. This variable isolation methodology allows any observed difference to be attributed with certainty solely to the effect of latency.

Configuration 1 — MAPPO_Baseline_NoLag_40M (ideal control case):

Agents receive environment observations in real time, without any artificial delay. This represents the ideal case in which network latency is absent.

```
python src/main.py --config=mappo --env-config=gymma \  
  with env_args.time_limit=500 \  
  env_args.key="rware:rware-tiny-2ag-v2" \  
  t_max=40000000 \  
  use_wandb=True \  
  wandb_project="Progetto-MAPPO-RWARE" \  
  wandb_team="emilianomazzone2000-university-of-catania" \  
  wandb_mode="online"
```

Configuration 2 — MAPPO_Lag2_40M (latency test case):

An artificial delay of 2 steps is injected into the observations received by the agents, simulating the network latency typical of a distributed control system. The key parameter distinguishing this configuration from the baseline is exclusively `env_args.obs_lag=2`.

```
python src/main.py --config=mappo --env-config=gymma \  
  with env_args.time_limit=500 \  
  env_args.key="rware:rware-tiny-2ag-v2" \  
  env_args.obs_lag=2 \  
  t_max=40000000 \  
  use_wandb=True \  
  wandb_project="Progetto-MAPPO-RWARE" \  
  wandb_team="emilianomazzone2000-university-of-catania" \  
  wandb_mode="online"
```

3.3 Parameter Description

- **--config=mappo:** Selects the MAPPO algorithm configuration file (`src/config/algs/mappo.yaml`).
- **--env-config=gymma:** Specifies the environment wrapper, compatible with Gym-based libraries such as RWARE.
- **env_args.time_limit=500:** Sets the maximum episode duration to 500 steps. Both configurations share this limit.

- **env_args.key:** Uniquely identifies the environment variant: rware-tiny-2ag-v2 is the minimal 2-agent configuration.
- **env_args.obs_lag=2:** Critical parameter: introduces a 2-step delay on observations. Absent in the baseline, present in the test case. This is the single parameter distinguishing the two experiments.
- **t_max=40000000:** Total number of training steps (40 million), chosen to observe complete convergence.
- **use_wandb=True:** Enables real-time logging to Weights & Biases for metric tracking.
- **wandb_project / wandb_team:** Identify the project and team on W&B for run organisation.
- **wandb_mode="online":** Enables continuous synchronisation with the W&B server during training.

4. Results Analysis

4.1 Episode Efficiency and Dynamics

A first element of fundamental importance emerges from observation of the `test_ep_length_mean` graph: for both experimental configurations, the mean episode duration during testing remains constantly and rigidly fixed at 500 steps throughout the entire training run, without any appreciable deviation. This finding, seemingly trivial, carries in fact a crucial diagnostic significance for the overall interpretation of the results.

The absence of variation in episode length categorically excludes the possibility that the performance degradation observed in the `MAPPO_Lag2_40M` configuration is attributable to premature episode terminations — a phenomenon that would be observed were agents to frequently incur fatal collisions or undesired terminal states. On the contrary, both systems operate for the entire available time window. The performance deficit of the latency variant is therefore not a survival problem, but one of intrinsic logistical inefficiency: agents with lag have the same amount of time as baseline agents to complete deliveries, yet complete significantly fewer, revealing a structural inability to plan and coordinate effective actions in the presence of delayed information. This distinction is fundamental to correctly framing the nature of the problem: latency does not cause the physical collapse of the system, but silently erodes its operational capacity from within.

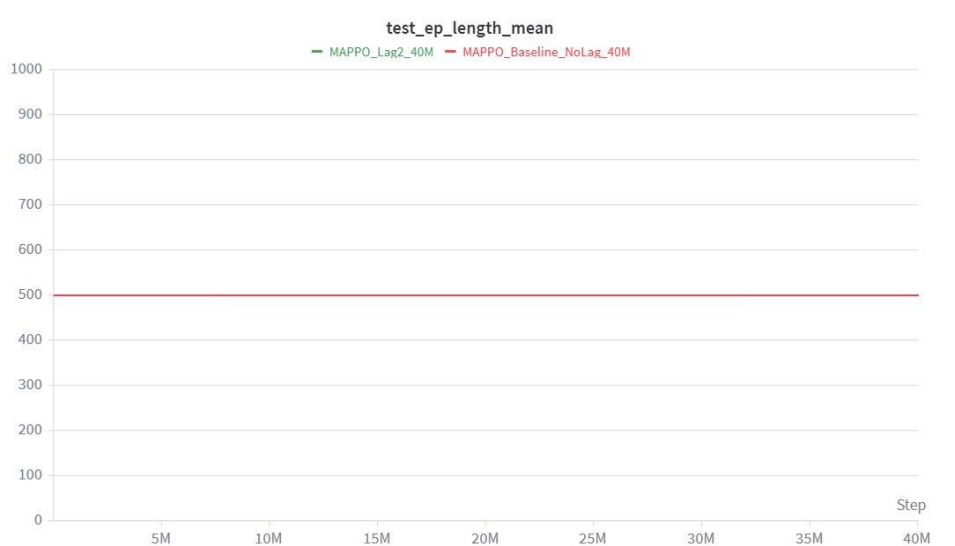


Figure 1 – test_ep_length_mean: episode duration is constantly fixed at 500 steps for both configurations throughout the entire training run.

4.2 Impact of Lag on Critic Convergence

The cross-analysis between test_return_mean and critic_loss reveals the central causal mechanism through which latency impairs learning, tracing two profoundly asymmetric evolutionary trajectories.

The MAPPO_Baseline_NoLag_40M configuration exhibits a coherent and orderly progression: the test_return_mean (red line) begins to grow consistently around 13–14 million steps, reaches the optimal plateau of approximately 2.0 around 20–25 million steps, and stabilises there with very low variance until the end of training. In perfect correlation with this trend, the baseline critic_loss progressively converges to zero: after an initial phase of high variability, it converges to values close to zero within 20–22 million steps and remains stable. This co-evolution describes a system in which the Centralised Critic rapidly acquires an accurate estimate of the value function, providing reliable training signals to the actors and enabling efficient policy learning.

The MAPPO_Lag2_40M configuration presents a radically different and pathological picture. The green critic_loss remains stubbornly high and highly noisy for a far longer interval — approximately until 28–30 million steps — with peaks exceeding values of 60, well above those recorded by the baseline during its most unstable phases. This persistence of the Critic error has a direct explanation in the latency mechanism: the Critic receives observations that are already stale at the time of their use, since the world state they describe belongs to a past 2 steps back. The consequence is systematic informational blindness: the Critic fails to build a coherent map between states and expected values, because the observed state and the actual state in which the agent acts are structurally decoupled. As long as this error remains high, the entire training signal is degraded. It is therefore unsurprising that the green test_return_mean remains anchored to low values and exhibits extremely slow growth until approximately 25–27 million steps, accumulating a learning delay of approximately 10–12 million steps relative to the baseline.

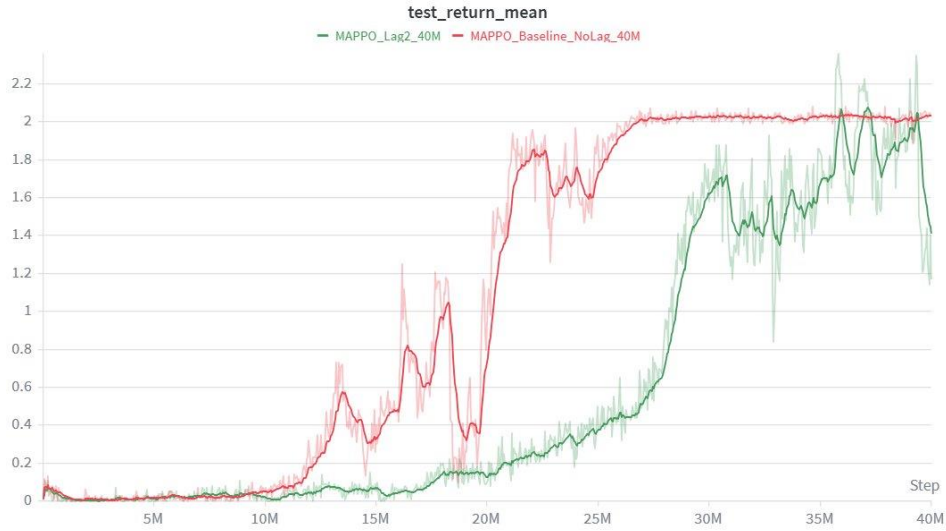


Figure 2 – test_return_mean: the baseline converges stably within 25M steps, while the lag variant accumulates a significant delay of approximately 10–12 million steps.

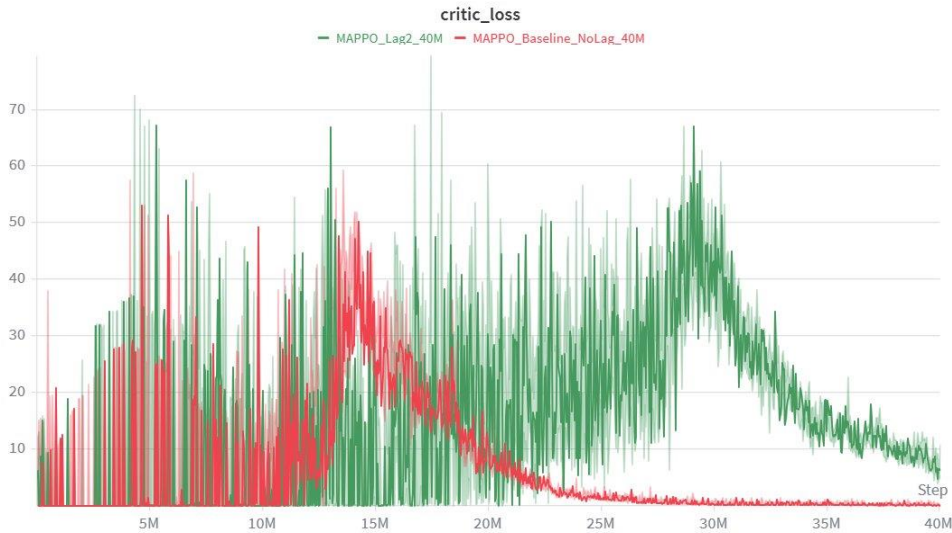


Figure 3 – critic_loss: the baseline loss converges to zero rapidly within 20–22M steps, while that of the lag variant remains high and noisy until approximately 30M steps, revealing the informational blindness of the Critic.

4.3 Instability, Gradient Shocks, and Policy Degradation

If the cross-analysis between return and critic loss reveals why the lag slows learning, the joint observation of test_return_mean and agent_grad_norm exposes its most severe consequence: the structural fragility of the learned policy and its tendency toward sudden collapse.

Throughout the entire training run, the gradient norm of the baseline agents remains contained and controlled, with rare and modest excursions above the threshold of 2.5, attesting to a stable optimisation process. The lag configuration instead presents a gradient profile characterised by anomalous spikes of disproportionate amplitude. The most prominent of these occurs around 14–15 million steps, where the green gradient norm reaches a value of approximately 28–29, nearly twenty times above the steady-state value. This spike occurs precisely at the moment when the lag system is

attempting its first transition towards higher performance, but encounters gradients so unstable as to render optimisation chaotic and undirected.

The most diagnostically relevant case, however, is that occurring around 36–37 million steps. At that depth of training, the MAPPO_Lag2_40M configuration had finally reached the optimal performance plateau of approximately 2.0 in test_return_mean, apparently aligning with the baseline after a far more turbulent learning trajectory. Yet the agent_grad_norm shows, at precisely that time window, a new anomalous spike of significant amplitude (approximately 6–7, well above the norm). The correlation with what occurs in test_return_mean is immediate and unambiguous: immediately following that spike, the return of the green line drops sharply from approximately 2.0 to values around 1.2–1.4, describing a catastrophic policy collapse that negates all the accumulated learning up to that point.

This event demonstrates with particular clarity the nature of the vulnerability introduced by latency. In an environment with delayed observations, the optimisation landscape perceived by the system is systematically distorted relative to the actual environment. A policy that appears to have converged to a local optimum may suddenly find itself exposed to a region of the parameter space where gradients are unstable and policy updates produce unpredictable and counterproductive effects. The baseline, having learned on coherent and timely observations, builds a robust policy that exhibits no analogous vulnerabilities; the lag, by contrast, introduces a latent fragility that can manifest at any time, rendering the system fundamentally unreliable even when its aggregate metrics appear to indicate satisfactory convergence.



Figure 4 – agent_grad_norm: the anomalous spikes of the lag variant (in particular at ~15M and ~37M steps) are directly correlated with the instabilities observed in test_return_mean; the spike at 37M steps triggers a catastrophic policy collapse.